

In []:

```
!pip install blockdiag
!mkdir diag
```

In [2]:

```
from IPython.display import Image
```

Contenedores en Python

- Listas, tuplas, conjuntos, diccionarios

In [2]:

```
a = [ 1, 2, 3 ]
b = ( 1, 2, 3 )
c = { 1, 2, 3 }
d = {}
type(d), type(a), type(b), type(c)
```

Out[2]:

```
(dict, list, tuple, set)
```

Aparentemente listas y tuplas funcionan de forma parecida.

In [3]:

```
print(a[1:])
for i in a:
    print(i)
if 3 in a:
    print('El 3 está')
```

```
[2, 3]
1
2
3
El 3 está
```

In [4]:

```
print(b[1:])
for i in b:
    print(i)
if 3 in b:
    print('El 3 está')
```

```
(2, 3)
1
2
3
El 3 está
```

Los conjuntos son ligeramente diferentes, porque no permiten indexar, pero sí iterar y determinar pertenencia. De hecho son especialmente eficientes en esta última operación.

In [5]:

```
# c[1:] <- No podemos
for i in c:
    print(i)
if 3 in c:
    print('El 3 está')
```

```
1
2
3
El 3 está
```

Los diccionarios se comportan como conjuntos, pero además tienen una operación de indexación especial, que permiten acceder a valores independientes de los elementos que actúan como índice.

In [5]:

```
d = { 1:'uno', 2:'dos', 3:'tres' }
for i in d:
    print(i, d[i])
if 3 in d:
    print('El 3 está')
```

```
1 uno
2 dos
3 tres
El 3 está
```

Conjuntos y diccionarios no preservan orden

Otra característica distintiva de conjuntos y diccionarios es que el orden no necesariamente refleja el orden en que se introdujeron los elementos. Se dice que son contenedores sin orden.

In [6]:

```
c = { 3, 1, 2 }
d = { 2:'dos', 1:'uno', 3:'tres' }
for i in c:
    print(i)
for i in d:
    print(i, d[i])
```

```
1
2
3
1 uno
2 dos
3 tres
```

In [33]:

```
s = 'askdgouhgasdfjahsdf'

def vocales(s):
    ret = ''
    for vocal in 'aeiouAEIOU':
        if vocal in s:
            ret += vocal
    return ret

vocales(s)
```

Out[33]:

'aou'

Mutabilidad en listas

Las listas tienen algunas características especiales, que deben ser conocidas para evitar sorpresas. Por ejemplo, se trata de un tipo *mutable*. Esto quiere decir que podemos usar el operador de indexación (corchetes) al lado izquierdo de una asignación.

In [39]:

```
a = list(range(20))  
print(id(a), a)  
a[19] = 58  
a[5:12] = []  
print(id(a), a)  
  
L = [1,2,3,4]  
L.append(5)  
  
dir(L)
```

```
1919581434440 [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
1919581434440 [0, 1, 2, 3, 4, 12, 13, 14, 15, 16, 17, 18, 58]
```

Out[39]:

```
['__add__',
 '__class__',
 '__contains__',
 '__delattr__',
 '__delitem__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattr__',
 '__getitem__',
 '__gt__',
 '__hash__',
 '__iadd__',
 '__imul__',
 '__init__',
 '__init_subclass__',
 '__iter__',
 '__le__',
 '__len__',
 '__lt__',
 '__mul__',
 '__ne__',
 '__new__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
 '__reversed__',
 '__rmul__',
 '__setattr__',
 '__setitem__',
 '__sizeof__',
 '__str__',
 '__subclasshook__',
 'append',
 'clear',
 'copy',
 'count',
 'extend',
 'index',
 'insert',
 'pop',
 'remove',
 'reverse',
 'sort']
```

In [41]:

```
s=list('abcdefghijk')
s[2:4]=''

help(L.count)
```

Help on built-in function count:

count(...) method of builtins.list instance
L.count(value) -> integer -- return number of occurrences of value

In [13]:

```
from string import ascii_letters as s

s = 'áaaaAáaaa'

def encontrar_vocales(s):
    ret = ''
    for letra in s:
        if letra in 'aeiouAEIOUáéíóúü':
            ret = ret + letra
    return ''.join(set(ret))

encontrar_vocales(s)
```

Out[13]:

'aáA'

La mutabilidad tiene consecuencias muy importantes. Vamos a ver una serie de ejemplos para ilustrarlas.

In [4]:

```
consonantes = ['b', 'c', 'd', 'f']
vocales = ['a', 'e', 'i']
letras = [consonantes, vocales]
letras2 = [['b', 'c', 'd', 'f'], ['a', 'e', 'i']]
```

Aparentemente letras y letras2 son lo mismo.

In [5]:

```
print('letras =', letras)
print('letras2 =', letras2)
print(letras == letras2)

letras = [['b', 'c', 'd', 'f'], ['a', 'e', 'i']]
letras2 = [['b', 'c', 'd', 'f'], ['a', 'e', 'i']]
True
```

Pero fíjate en el resultado de manipular la lista vocales sin tocar en absoluto la lista letras .

In [6]:

```
vocales.append('o')
print('letras =', letras)
print('letras2 =', letras2)
print(letras == letras2)
```

```
letras = [['b', 'c', 'd', 'f'], ['a', 'e', 'i', 'o']]
letras2 = [['b', 'c', 'd', 'f'], ['a', 'e', 'i']]
False
```

Python no guarda el contenido de las listas `consonantes` y `vocales` como elementos de `letras`, sino que guarda una referencia al objeto original, que puede manipularse afectando a todas las variables que contienen referencias a esas listas.

Podemos ver que se trata de objetos diferentes empleando la función `id` o bien con [PythonTutor](http://www.pythontutor.com/visualize.html#code=consonantes+%3D+%5B'b',+'c',+'d',+'f'%5D%0Avocales+%3D+%5B'a',+'e',+'i',+'o'%5D&cumulative=false&heapPrimitives=false&textReferences=false&py=2&rawInputLstJSON=%5B%5C) (<http://www.pythontutor.com/visualize.html#code=consonantes+%3D+%5B'b',+'c',+'d',+'f'%5D%0Avocales+%3D+%5B'a',+'e',+'i',+'o'%5D&cumulative=false&heapPrimitives=false&textReferences=false&py=2&rawInputLstJSON=%5B%5C>

In [11]:

```
(id(letras), id(letras2))
```

```
912297503944 912316674440
```

Esto tiene más implicaciones de las que vemos a primera vista, porque el paso de parámetros no es más que un caso particular de todo esto.

In [12]:

```
def f(lista):
    lista[1].append('u')

f(letras2)

print(letras2)
```

```
[['b', 'c', 'd', 'f'], ['a', 'e', 'i', 'u']]
```

Hasta ahora cuando pasábamos un valor a una función se trataba de una copia, que podía manipular a su antojo sin afectar al programa que llamaba. Por ejemplo:

In [13]:

```
def cifras(n):  
    while n > 0:  
        print(n%10)  
        n //= 10
```

```
n = 1985  
cifras(n)  
print(n)
```

```
5  
8  
9  
1  
1985
```

Al pasar una lista se está pasando una referencia al objeto correspondiente. Al tratarse de un objeto mutable la función puede devolver resultados a la función que llama sin ni siquiera usar `return`. Nunca hagas esto.

In [14]:

```
def cifras(n, lista): # Nunca hagas esto  
    while n > 0:  
        lista.append(n%10)  
        n //= 10
```

```
n = []  
cifras(1985, n)  
print(n)
```

```
[5, 8, 9, 1]
```

Recuerda que programas para que otros lean tus programas. Si escondes el valor de retorno solo estás dificultando la lectura.

In [15]:

```
def cifras(n):  
    lista = []  
    while n > 0:  
        lista.append(n%10)  
        n //= 10  
    return lista
```

```
print(cifras(1985))
```

```
[5, 8, 9, 1]
```


Slicing en listas

Se llama *slicing* (partir en rodajas) a las operaciones que seleccionan una parte de la lista, generando una nueva lista en principio más pequeña. Familiarízate con las operaciones de *slicing*, son las más frecuentes y no solo en listas.

Veamos algunos ejemplos.

In [16]:

```
L1 = list(range(10))
L2 = list(range(10))
print(L1)
print(L2)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Cuando se selecciona una parte de la lista el resultado es una lista. En cambio si se usa un índice concreto (operación de indexación) el resultado es un elemento concreto. Eso es muy importante para manipular la lista. Veamos dos ejemplos en los que sustituimos el segundo elemento.

En el primer caso usamos indexación, metemos como segundo elemento el resultado de la operación de slicing `L1[5:]`. Es decir, a partir del elemento 5.

En el segundo caso sustituimos la lista `L2[1:1]` por la lista `L2[5:]`. Aunque `L2[1:1]` solo tenga un elemento sigue siendo una lista, no un elemento. Por eso al sustituir una lista por otra estamos insertando.

In [17]:

```
L1[1] = L1[5:]
L2[1:1] = L2[5:]
print(L1)
print(L2)
```

```
[0, [5, 6, 7, 8, 9], 2, 3, 4, 5, 6, 7, 8, 9]
[0, 5, 6, 7, 8, 9, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Un ejercicio interesante es practicar *slicing* para seleccionar las distintas partes de un Sudoku.

In [18]:

```
sudoku = [
[4,9,1,3,6,2,8,7,5],
[5,2,6,8,7,1,4,9,3],
[7,8,3,9,5,4,6,1,2],
[2,3,9,4,1,7,5,8,6],
[1,5,7,6,3,8,9,2,4],
[6,4,8,2,9,5,7,3,1],
[8,7,2,1,4,6,3,5,9],
[9,1,4,5,8,3,2,6,7],
[3,6,5,7,2,9,1,4,8]]
```

Practica con filas, y con elementos de una fila. Pero hasta que domines las *list comprehensions* que se cuentan más adelante no serás capaz de seleccionar los cuadrantes.

Métodos de una lista

Familiarízate con los métodos de las listas. Se utilizan muchísimo.

In [7]:

```
L = list(range(10))
```

Añadir un elemento al final.

In [8]:

```
L.append(4)  
L
```

Out[8]:

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 4]
```

Contar todas las apariciones del elemento 4 .

In [9]:

```
L.count(4)
```

Out[9]:

```
2
```

Insertar el elemento 80 en la posición 3 .

In [10]:

```
L.insert(3, 80)  
L
```

Out[10]:

```
[0, 1, 2, 80, 3, 4, 5, 6, 7, 8, 9, 4]
```

Añadir al final los elementos de otra lista.

In [23]:

```
L.extend([2,3,4])  
L
```

```
[0, 1, 2, 80, 3, 4, 5, 6, 7, 8, 9, 4, 2, 3, 4]
```

Eliminar la primera ocurrencia del elemento 4 .

In [24]:

```
L.remove(4)  
L
```

```
[0, 1, 2, 80, 3, 5, 6, 7, 8, 9, 4, 2, 3, 4]
```

Imprimir la posición del primer elemento de valor 4 .

In [25]:

```
L.index(4)
```

```
10
```

Eliminar el último de la lista devolviendo su valor. También se puede indicar una posición para eliminar uno cualquiera de la lista.

In [26]:

```
L.pop()
```

```
4
```

Ordenar los elementos de la lista en orden creciente.

In [27]:

```
L.sort()  
L
```

```
[0, 1, 2, 2, 3, 3, 4, 5, 6, 7, 8, 9, 80]
```

Invertir el orden de todos los elementos de la lista.

In [28]:

```
L.reverse()  
L
```

```
[80, 9, 8, 7, 6, 5, 4, 3, 3, 2, 2, 1, 0]
```

Clonado de listas

Al modificar el contenido de la lista se puede afectar al recorrido de la lista. Por ejemplo, considera la siguiente función:

In [1]:

```
def borraDuplicados(L1, L2): # ¡OJO! ¡Esta función es incorrecta!
    '''Asume que L1 y L2 son listas.
    Elimina todos los elementos de L1 que estén presentes en L2.'''
    for e in L1:
        if e in L2:
            L1.remove(e)
```

Veamos un posible uso de la función.

In [2]:

```
L1 = [1,2,3,4]
L2 = [1,2,5,6]
borraDuplicados(L1, L2)
print('L1 =', L1)
```

```
L1 = [2, 3, 4]
```

¡Sorpresa! El número 2 está presente en L2 pero no es eliminado de L1 . ¿Qué ha pasado?

El motivo es que el recorrido de la lista se realiza internamente con un contador que va desde el 0 (primer elemento) hasta len(L1) (sin incluirlo). En la primera iteración comprueba el elemento 0 y descubre que es un duplicado. Por tanto lo elimina, pero al eliminarlo el primer elemento deja de existir y su lugar es ocupado por el segundo elemento. El bucle for no tiene forma de saber que se ha cambiado el orden de los elementos y sigue por donde iba, por el segundo elemento. Pero el que ahora ocupa el segundo lugar es el que antes era el tercero. Se ha saltado el 2 .

La lección a recordar es que la mutación de una lista invalida los iteradores. Todos los for que recorran la lista y que se estén ejecutando en el momento de la mutación dejan de tener sentido.

Por tanto el recorrido debe separarse de la mutación, debe hacerse sobre objetos distintos. La forma más sencilla es clonando la lista. Es decir, creando otra lista con los mismos elementos. Y eso ya sabemos hacerlo:

In [32]:

```
def borraDuplicados(L1, L2):
    '''Asume que L1 y L2 son listas.
    Elimina todos los elementos de L1 que estén presentes en L2.'''
    for e in L1[:]:
        if e in L2:
            L1.remove(e)
```

La expresión L1[:] es una nueva lista que contiene todos los elementos de L1 . Ahora el recorrido se hace sobre esa nueva lista, mientras que la operación de mutación remove se realiza sobre la lista L1 original.

In [33]:

```
L1 = [1,2,3,4]
L2 = [1,2,5,6]
borraDuplicados(L1, L2)
print('L1 =', L1)
```

```
L1 = [3, 4]
```

El mismo resultado se obtiene con la llamada a la función `list(L1)` . Crea una nueva lista con los elementos de la que se pasa como argumento.

In [34]:

```
def borraDuplicados(L1, L2):
    '''Asume que L1 y L2 son listas.
    Elimina todos los elementos de L1 que estén presentes en L2.'''
    for e in list(L1):
        if e in L2:
            L1.remove(e)
```

Ambas opciones son perfectamente razonables en un programa Python. Cuál usar es un tema de gusto personal.

In [35]:

```
L1 = [1,2,3,4]
L2 = [1,2,5,6]
borraDuplicados(L1, L2)
print('L1 =', L1)
```

```
L1 = [3, 4]
```

¿Y si los elementos de la lista son a su vez mutables? Volvemos a tener el mismo problema. Al copiar los elementos cada uno de ellos debe crearse una nueva copia de su contenido. Veamos un ejemplo.

In [13]:

```
frutas = [ 'pera', 'manzana', 'naranja' ]
verduras = [ 'tomate', 'apio', 'puerro' ]
productos = [ frutas, verduras ]

productos2 = productos[:]
frutas.append('melon')
productos2
```

Out[13]:

```
[['pera', 'manzana', 'naranja', 'melon'], ['tomate', 'apio', 'puerro']]
```

La forma más sencilla de realizar copias de objetos complejos es mediante la biblioteca `copy` .

In [12]:

```
from copy import deepcopy

productos2 = deepcopy(productos)
verduras.append('calabaza')
productos2
```

Out[12]:

```
[['pera', 'manzana', 'naranja', 'melon'], ['tomate', 'apio', 'puerro']]
```

Ahora las dos estructuras de datos son completamente independientes y al alterar una o parte de ella no notamos cambio alguno en la otra.

Comprensiones de listas

Una construcción muy importante en Python es la denominada *list comprehension* o comprensión de lista. Se trata de una notación compacta para generar listas (u otros contenedores) cuyos elementos se puedan escribir en forma de expresiones con los elementos de otra lista.

Por ejemplo, una lista con los primeros 10 cuadrados de números naturales.

In [38]:

```
[ x**2 for x in range(1,11) ]
```

Out[38]:

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Comprensiones de lista con filtros

- Solo se incluyen los que cumplen la condición.
 - Ejemplo: `os.listdir` dice el contenido de una carpeta.

In [10]:

```
import os
os.listdir('diag')
```

Out[10]:

```
['biseccion.bd',
 'biseccion.png',
 'biseccion.svg',
 'for.bd',
 'for.png',
 'for.svg',
 'fun-call.bd',
 'fun-call.png',
 'fun-call.svg',
 'if-elif-else.bd',
 'if-elif-else.png',
 'if-elif-else.svg',
 'if-else.bd',
 'if-else.png',
 'if-else.svg',
 'if.bd',
 'if.png',
 'if.svg',
 'lambda-abstr.bd',
 'lambda-abstr.png',
 'lambda-abstr.svg',
 'maxfn.png',
 'maxfn.svg',
 'maxfncall.png',
 'maxfncall.svg',
 'maxfncall2.png',
 'maxfncall2.svg',
 'seq.bd',
 'seq.png',
 'seq.svg',
 'while.bd',
 'while.png',
 'while.svg']
```

También se puede añadir una condición que actúa como filtro. Es decir, de los elementos generados solo los que cumplen la condición se incluyen en la lista. Por ejemplo, tenemos la función `os.listdir` que nos dice el contenido de una carpeta. Veamos que pasa si la llamo en el directorio de los diagramas de la asignatura.

Hay archivos que acaban en `.bd` que son diagramas y otros `.png` que son el resultado de dibujarlos. Veamos, por ejemplo, los diagramas que muestran varias formas de `if`.

In [16]:

```
[ f for f in os.listdir('diag') if f.endswith('.bd') and f.startswith('if') ]
```

Out[16]:

```
['if-elif-else.bd', 'if-else.bd', 'if.bd']
```

Sudoku

- Vamos a seleccionar partes de él ahora que sabemos todo acerca de las *list comprehensions*.

In [1]:

```
sudoku = [  
[4,9,1,3,6,2,8,7,5],  
[5,2,6,8,7,1,4,9,3],  
[7,8,3,9,5,4,6,1,2],  
[2,3,9,4,1,7,5,8,6],  
[1,5,7,6,3,8,9,2,4],  
[6,4,8,2,9,5,7,3,1],  
[8,7,2,1,4,6,3,5,9],  
[9,1,4,5,8,3,2,6,7],  
[3,6,5,7,2,9,1,4,8]]
```

In [18]:

```
[fila[3:6] for fila in sudoku[3:6]]
```

Out[18]:

```
[4, 1, 7, 6, 3, 8, 2, 9, 5]
```

Seleccionemos el cuadrante superior izquierdo.

In [18]:

```
[fila[:3] for fila in sudoku[:3]]
```

Out[18]:

```
[[4, 9, 1], [5, 2, 6], [7, 8, 3]]
```

Ahora el inferior derecho.

In [19]:

```
[s[6:] for s in sudoku[6:]]
```

Out[19]:

```
[[3, 5, 9], [2, 6, 7], [1, 4, 8]]
```

Ahora el del medio.

In [20]:

```
[s[3:6] for s in sudoku[3:6]]
```

Out[20]:

```
[[4, 1, 7], [6, 3, 8], [2, 9, 5]]
```


Ahora la tercera columna.

In [21]:

```
[ s[2] for s in sudoku ]
```

Out[21]:

```
[1, 6, 3, 9, 7, 8, 2, 4, 5]
```

Practica tú. Haz por ejemplo que los cuadrantes aparezcan como una lista simple en lugar de una lista de listas.

Piénsalo un poco por tí mismo y si no llegas a la solución sigue leyendo. A partir de aquí se puede considerar un uso avanzado de las *list comprehensions*. No te preocupes si no las entiendes.

El cuadrante superior izquierdo como lista simple:

In [15]:

```
[ i for fila in sudoku[:3] for i in fila[:3] ]
```

Out[15]:

```
[4, 9, 1, 5, 2, 6, 7, 8, 3]
```

In [17]:

```
[4,9,1] + [5,2,6]
```

Out[17]:

```
[4, 9, 1, 5, 2, 6]
```

El cuadrante inferior derecho como lista simple:

In [23]:

```
[ i for s in sudoku[6:] for i in s[6:] ]
```

Out[23]:

```
[3, 5, 9, 2, 6, 7, 1, 4, 8]
```

El del medio como lista simple:

In [24]:

```
[ i for s in sudoku[3:6] for i in s[3:6] ]
```

Out[24]:

```
[4, 1, 7, 6, 3, 8, 2, 9, 5]
```

Elabora una función para devolver el cuadrante (x, y) siendo x e y números entre 0 y 2.

In [21]:

```
def cuadrante(sudoku, x, y):  
    return [ i for s in sudoku[3*y:][:3] for i in s[3*x:][:3] ]
```

Veamos para probarlo los cuadrantes centrales.

In [22]:

```
cuadrante(sudoku, 0, 1)
```

Out[22]:

```
[2, 3, 9, 1, 5, 7, 6, 4, 8]
```

In [28]:

```
cuadrante(sudoku, 1, 1)
```

Out[28]:

```
[4, 1, 7, 6, 3, 8, 2, 9, 5]
```

In [29]:

```
cuadrante(sudoku, 2, 1)
```

Out[29]:

```
[5, 8, 6, 9, 2, 4, 7, 3, 1]
```

Iteradores

- Objeto que encapsula recorrido de un contenedor.
 - `iter` devuelve iterador, `next` accede al elemento actual.

In [15]:

```
L = [1, 5, 2, 0]  
i = iter(L)
```

In [22]:

```
i1,i2 = iter(L), iter(L)
```

In [25]:

```
next(i2)
```

Out[25]:

```
1
```

In [2]:

```
iterador = iter(sudoku)
```

In [3]:

```
next(iterador)
```

Out[3]:

```
[4, 9, 1, 3, 6, 2, 8, 7, 5]
```

Generadores

- Pueden detener *temporalmente* su ejecución (`yield`).

In [27]:

```
def gen():  
    for i in range(5):  
        yield i**3
```

In [29]:

```
gen()
```

Out[29]:

```
<generator object gen at 0x0000017114969468>
```

In [30]:

```
for i in gen():  
    print(i)
```

```
0  
1  
8  
27  
64
```

Pueden iterarse como un contenedor, con `for` o directamente con `iter` y `next` .

In [31]:

```
it1, it2 = iter(gen()), iter(gen())
```

In [36]:

```
next(it2)
```

Out[36]:

```
1
```

Pero son mucho más eficientes que las listas con o sin comprensiones.

In [38]:

```
def gen_infinito():  
    i = 0  
    while True:  
        yield i**3  
        i += 1
```

In [40]:

```
it = iter(gen_infinito())  
for i in range(5):  
    print(next(it))
```

0
1
8
27
64

In [41]:

```
def fib():  
    a, b = 1, 1  
    yield a  
    while True:  
        yield b  
        a, b = b, a+b
```

In [43]:

```
it = iter(fib())  
for i in range(10):  
    print(next(it))
```

1
1
2
3
5
8
13
21
34
55

Expresiones generadoras

- Son a generadores lo que *list comprehensions* son a listas

In [58]:

```
gen = (x**3 for x in range(5))
```

In [57]:

```
for i in gen:
    print(i)
```

0
1
8
27
64

- No ocupan memoria, solo el último generado

In [47]:

```
gen = (x**3 for x in range(5000000000000000))
it = iter(gen)
[next(it) for i in range(5)]
```

Out[47]:

[0, 1, 8, 27, 64]

Conversiones y construcción

- Un contenedor puede construirse a partir de un iterable

In [44]:

```
list('abcde'), list(range(5))
```

Out[44]:

(['a', 'b', 'c', 'd', 'e'], [0, 1, 2, 3, 4])

In [47]:

```
def cube(L): return (x**3 for x in L)
tuple(cube(range(5)))
```

Out[47]:

(0, 1, 8, 27, 64)

In [49]:

```
palabras = ['uno', 'dos', 'tres']
numpal = ((i, palabras[i-1]) for i in [1,2,3])
dict(numpal)
```

Out[49]:

{1: 'uno', 2: 'dos', 3: 'tres'}

Generadores incluidos

In [41]:

```
s,t = 'ABCDE', 'zyxwv'
```

In [40]:

```
list(enumerate(s))
```

Out[40]:

```
[(0, 'A'), (1, 'B'), (2, 'C'), (3, 'D'), (4, 'E')]
```

In [51]:

```
list(zip(s,t))
```

Out[51]:

```
[('A', 'z'), ('B', 'y'), ('C', 'x'), ('D', 'w'), ('E', 'v')]
```

In [50]:

```
dict(zip(s,t))
```

Out[50]:

```
{'A': 'z', 'B': 'y', 'C': 'x', 'D': 'w', 'E': 'v'}
```

Iteración revisada

In [33]:

```
L = L1 = L2 = []  
cond = False  
def do_something(x=None, y=None): pass
```

In [34]:

```
for i in L:                # Sabemos qué iterar  
    do_something(i)
```

In [35]:

```
for pos,i in enumerate(L): # Necesitamos posiciones  
    do_something(pos, i)
```

In [36]:

```
for a,b in zip(L1,L2):     # Varios iterables sincronizados  
    do_something(a,b)
```

In [37]:

```
while cond:                # General, ¡necesita f. decr.!  
    do_something()
```

In [44]:

```
[[1,0,0],  
 [0,1,0],  
 [0,0,1]]
```

Out[44]:

```
[[1, 0, 0], [0, 1, 0], [0, 0, 1]]
```

In []:

```
def mul(A,B):
```

Sucesión de Fibonacci

In [59]:

```
def f():  
    for i in range(5):  
        yield i**3
```

In []:

In [23]:

```
def fib():  
    yield 0  
    yield 1  
    for f1, f2 in zip(fib(), fib1()):  
        yield f1 + f2  
  
def fib1():  
    it = fib(); next(it)  
    yield from it
```

In [24]:

```
f = fib()  
[next(f) for i in range(10)]
```

Out[24]:

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Triángulo de Pascal

El triángulo de Pascal es un triángulo de números que se construye de la siguiente manera. Empieza con un 1 en la fila superior. Cada fila posterior se construye a partir de la anterior, donde el primer y el último número de cada fila son iguales que el primer y último número de la anterior, y cada número entre ellos es la suma de los dos a la izquierda y a la derecha en la fila inmediatamente superior.

```
    1
   1 1
  1 2 1
 1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
```

In [1]:

```
def triangulo_pascal(n):
    L = [' '.join([str(i) for i in x]) for x in pascal(n)]
    for s in L:
        print('{:^'+str(len(L[-1]))+'}'.format(s))
```

Como siempre utilizamos una estrategia top-down. Asumimos que los números están calculados y los imprimimos como dice el enunciado. En esta solución aprovecha el [método format](#)

(<https://docs.python.org/3/library/stdtypes.html#str.format>) de las cadenas. La expresión

'{: ^30}'.format(s) es una cadena de 30 caracteres que contiene la cadena s centrada y todos los demás caracteres como espacios. En este caso el ancho de la cadena es el ancho de la última fila, por lo que habrá que calcularlo.

Por supuesto, también se puede hacer escribiendo los espacios a mano.

In [2]:

```
def triangulo_pascal(n):
    L = [' '.join([str(i) for i in x]) for x in pascal(n)]
    for s in L:
        espacios = (len(L[-1]) - len(s)) // 2
        print(' '*espacios + s)
```

Ya solo queda calcular los números correspondientes a cada línea.

In [3]:

```
def pascal(n):
    if n == 1: return [[1]]
    L = pascal(n-1)
    return L + [linea_siguiete(L[-1])]

def linea_siguiete(L):
    return [a+b for a,b in zip(L+[0], [0]+L)]
```


Es todavía mas simple usar un generador para producir los números.

In [4]:

```
def pascal(n):  
    L = [1]  
    for i in range(n):  
        yield L  
        L = linea_siguiiente(L)
```

In [5]:

```
triangulo_pascal(6)
```

```
    1  
   1 1  
  1 2 1  
 1 3 3 1  
1 4 6 4 1  
1 5 10 10 5 1
```

Sumar días a una fecha

Elaborar una función `sumar_dias` que devuelva la fecha que corresponde a incrementar la dada en un número de días.

Es un caso típico de *divide y vencerás* pero puede orientarse como un problema de recorrido de secuencias. Se trata de construir la secuencia que me permite saber el día, mes y año resultante con comodidad.

Puede hacerse de forma iterativa, empezando por el conocimiento básico (días por mes), adaptando ese conocimiento al caso concreto (días por mes desde uno dado), y generando los valores que facilitan el cálculo (sumas acumuladas de los días hasta el comienzo y fin del mes).

In [35]:

```
from itertools import accumulate, dropwhile, repeat

def sumar_dias(fecha, dias):
    '''Asume fecha tupla en formato (dia, mes, año), dias entero.
    Devuelve la fecha correspondiente a la indicada en fecha incrementada en el número
    de días indicado en dias.'''
    dia, mes, año = fecha
    dias += dia
    dias_min, dias_max, mes, año = next(dropwhile(lambda x: x[1] < dias, dias_hasta_mes_desde(mes, año)))
    return dias - dias_min, mes, año

def dias_hasta_mes_desde(mes_desde, año_desde):
    '''Asume mes_desde, año_desde enteros.
    Genera tuplas (dias_min, dias_max, mes, año) desde el mes_desde y año_desde indicados, indefinidamente.
    Los valores de dias_min y dias_max son los días que faltan para el comienzo y fin del mes respectivamente,
    contados desde el comienzo del mes_desde y año_desde indicados.'''
    por_mes = ((0,d,m,a) for d,m,a in dias_por_mes_desde(mes_desde, año_desde))
    yield from accumulate(por_mes, lambda a,b: (a[1], a[1] + b[1], b[2], b[3]))

def dias_por_mes_desde(mes_desde, año_desde):
    '''Asume mes_desde, año_desde enteros.
    Genera tuplas (dias_mes,mes,año) de los meses a partir del mes y año indicados, indefinidamente.'''
    dias_mes = dropwhile(lambda x: mes(x) != mes_desde, dias_por_mes_desde_año(año_desde))
    yield from dias_mes

def dias_por_mes_desde_año(año_desde):
    '''Asume año_desde entero.
    Genera tuplas (dias_mes, mes, año) desde enero del año_desde, indefinidamente.'''
    ndias = ((31,28,31,30,31,30,31,31,30,31,30,31),
              (31,29,31,30,31,30,31,31,30,31,30,31))
    while True:
        yield from zip(ndias[es_bisiesto(año_desde)], range(1,13), repeat(año_desde))
        año_desde += 1

def es_bisiesto(año):
    '''Asume año entero.
    Devuelve True si año es bisiesto y False en caso contrario.'''
    return año % 400 == 0 or año % 100 != 0 and año % 4 == 0
```

Algunas pruebas para verificar que todo funciona como debe.

In [33]:

```
it = iter(dias_por_mes_desde_año(2016))
print('dias\tmes\taño')
for i in range(12):
    print(*next(it), sep='\t')
```

dias	mes	año
31	1	2016
29	2	2016
31	3	2016
30	4	2016
31	5	2016
30	6	2016
31	7	2016
31	8	2016
30	9	2016
31	10	2016
30	11	2016
31	12	2016

In [32]:

```
it = iter(dias_por_mes_desde(5,2016))
print('dias\tmes\taño')
for i in range(12):
    print(*next(it), sep='\t')
```

dias	mes	año
31	5	2016
30	6	2016
31	7	2016
31	8	2016
30	9	2016
31	10	2016
30	11	2016
31	12	2016
31	1	2017
28	2	2017
31	3	2017
30	4	2017

In [31]:

```
it = iter(dias_hasta_mes_desde(5,2016))
print('min\tmax\tmes\taño')
for i in range(12):
    print(*next(it), sep='\t')
```

min	max	mes	año
0	31	5	2016
31	61	6	2016
61	92	7	2016
92	123	8	2016
123	153	9	2016
153	184	10	2016
184	214	11	2016
214	245	12	2016
245	276	1	2017
276	304	2	2017
304	335	3	2017
335	365	4	2017

In [38]:

```
sumar_dias((1,5,2016), 30)
```

Out[38]:

(31, 5, 2016)

In [25]:

```
sumar_dias((21,1,2016), 1)
```

Out[25]:

(22, 1, 2016)

In [26]:

```
sumar_dias((31,1,2016), 1)
```

Out[26]:

(1, 2, 2016)

In [27]:

```
sumar_dias((31,1,2016), 0)
```

Out[27]:

(31, 1, 2016)

In [28]:

```
sumar_dias((21,1,2016), 366)
```

Out[28]:

(21, 1, 2017)

In []: